Microsoft Plugin Architecture: reuse the core, govern each host

Contents

Executive Summary

1. Choose the host route

2. Reuse the core, not the wrapper

3. Preserve supported paths

4. Govern a small portfolio

Citations

Executive Summary

Bottom line: standardize skills and MCP, then govern compatibility and ownership for each host.

Audience: Executive decision-makers

Decision: Standardize skills and MCP; govern compatibility and ownership per host.

Narration

Bottom line: standardize skills and MCP, then govern compatibility and ownership for each host.

Cowork uses a Microsoft 365 app package. Declarative agents use MCP or REST API actions. GitHub Copilot uses plugin packages with portable and client-specific parts.

Reuse skill logic and MCP configuration where tested. Keep authentication, manifests, discovery and client-only components inside each host package.

REST API actions remain supported in declarative agents. Agent Plugins 1.0 adds portability, while legacy GitHub Copilot plugins continue to load.

Assign one owner per capability and service. Publish a host compatibility matrix, version each package, and add routes only when demand justifies lifecycle cost.

1. Choose the host route

Cowork uses a Microsoft 365 app package. Declarative agents use MCP or REST API actions. GitHub Copilot uses plugin packages with portable and client-specific parts.

Claims

Three host routes use different delivery contracts: Cowork uses Microsoft 365 app packages; declarative agents use MCP or API actions; GitHub Copilot uses plugin packages.

Cowork packages skills and connectors in a standard Microsoft 365 app package.

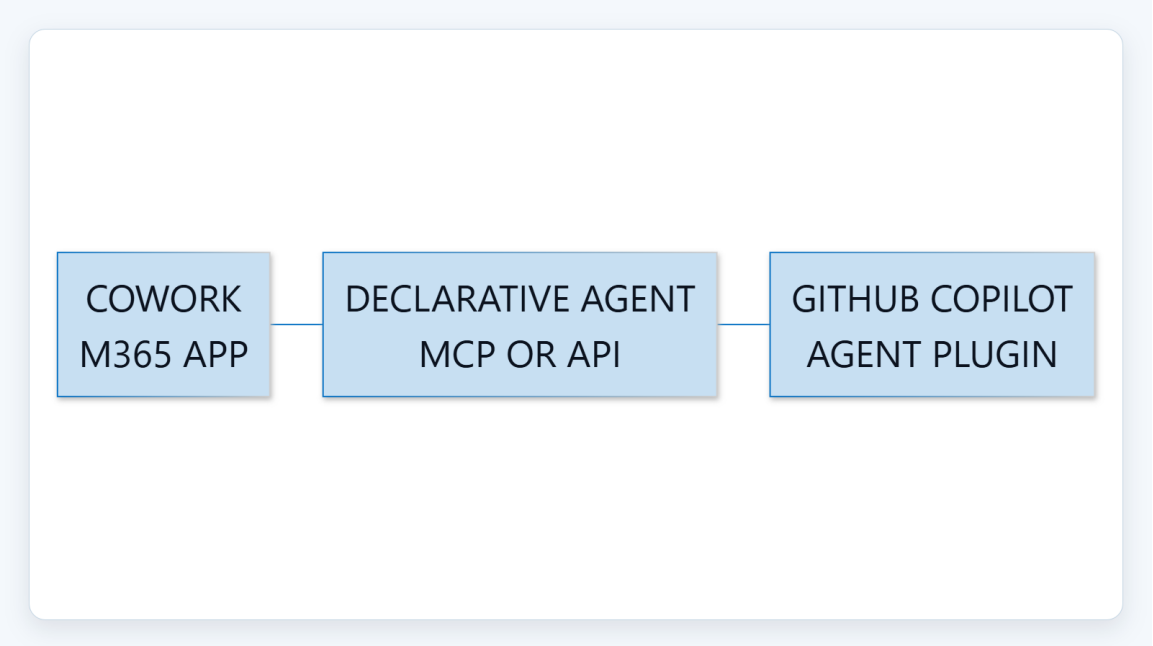
Microsoft 365 declarative agents support MCP and REST API plugins as in-agent actions; REST API tools are defined in the plugin manifest.

Speaker notes

Evidence: the three selected official sources, retrieved 2026-10-06. Decision implication: name the host and package contract for every portfolio item; do not describe these routes as interchangeable.

Assets and alt text

visual-three-routes: Three host routes share concepts, but each has a distinct delivery contract. (mermaid diagram)



2. Reuse the core, not the wrapper

Reuse skill logic and MCP configuration where tested. Keep authentication, manifests, discovery and client-only components inside each host package.

Claims

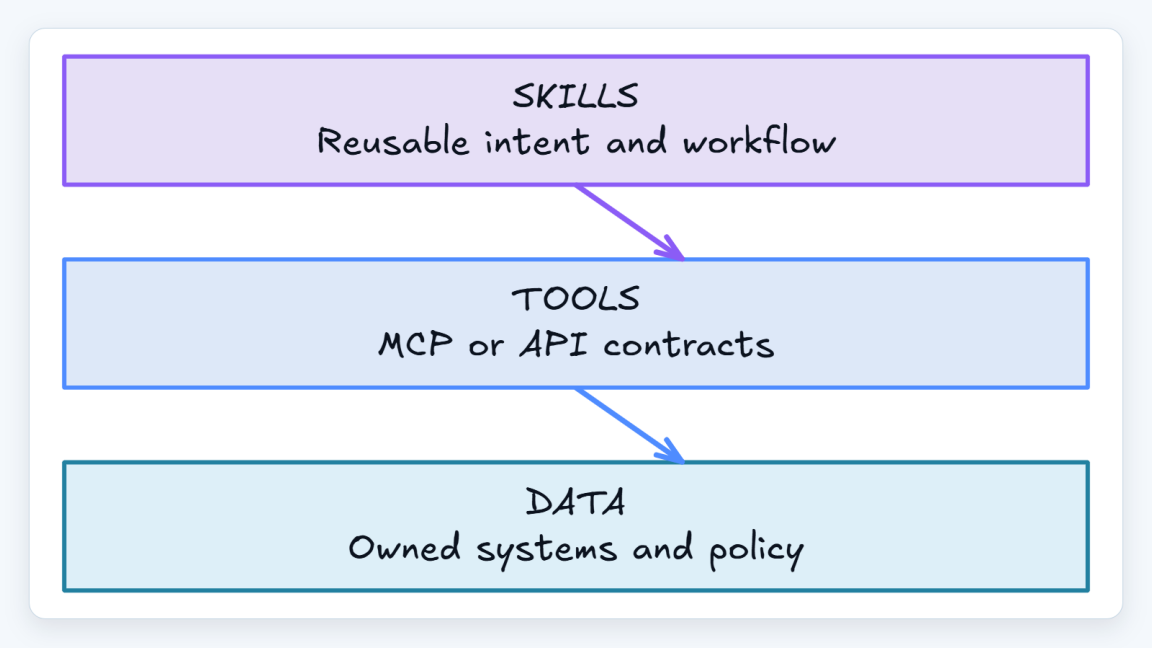
Skills and MCP configuration can be reusable, while manifests, discovery, authentication and client-specific components remain host-specific.

Speaker notes

Evidence: Cowork skill portability, declarative-agent MCP/API behavior, and GitHub Agent Plugins portability and legacy support. Interpretation: the reusable layer should be separated from host packaging and operations.

Assets and alt text

visual-reuse-boundary: A governed reusable core connects skills to tools and data while host wrappers own policy and packaging. (excalidraw diagram)



3. Preserve supported paths

REST API actions remain supported in declarative agents. Agent Plugins 1.0 adds portability, while legacy GitHub Copilot plugins continue to load.

Claims

Microsoft 365 declarative agents support MCP and REST API plugins as in-agent actions; REST API tools are defined in the plugin manifest.

GitHub Agent Plugins 1.0 ports skills and MCP configuration across compatible clients; client-specific components and legacy plugins remain supported.

Speaker notes

Evidence: Microsoft Learn documents REST API plugins as declarative-agent actions; GitHub documents both Agent Plugins 1.0 and legacy Copilot plugins as supported.

Assets and alt text

visual-supported-paths: Modern portability does not make supported REST API or legacy plugin routes obsolete. (marp diagram)



4. Govern a small portfolio

Assign one owner per capability and service. Publish a host compatibility matrix, version each package, and add routes only when demand justifies lifecycle cost.

Claims

Three host routes use different delivery contracts: Cowork uses Microsoft 365 app packages; declarative agents use MCP or API actions; GitHub Copilot uses plugin packages.

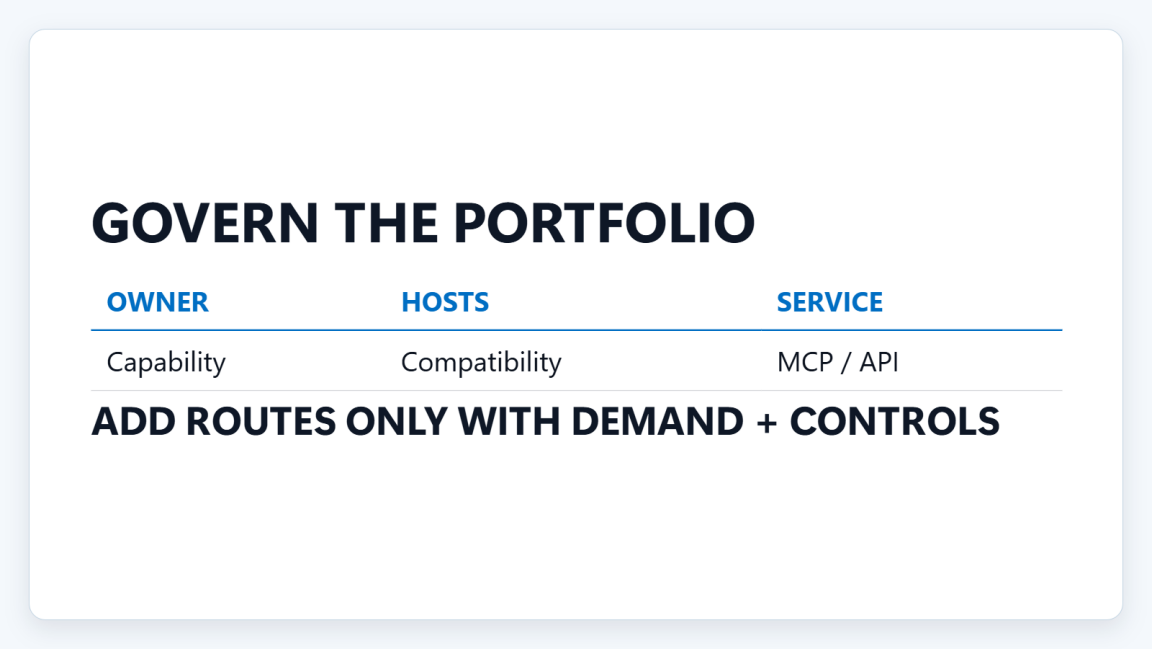
Skills and MCP configuration can be reusable, while manifests, discovery, authentication and client-specific components remain host-specific.

Speaker notes

Proposed operating model, grounded in the documented route and compatibility boundaries. No source prescribes an enterprise org chart or ROI target.

Assets and alt text

visual-portfolio: A small governed portfolio makes compatibility and operational ownership explicit. (marp diagram)



Citations

[ev-01, ev-02, ev-03] Build plugins for Copilot Cowork (retrieved 2026-10-06T00:46:44.482Z)

[ev-04, ev-05, ev-06] MCP and API plugins for declarative agents (retrieved 2026-10-06T00:46:44.749Z)

[ev-07, ev-08, ev-09] About GitHub Copilot plugins (retrieved 2026-10-06T00:46:44.903Z)